

Lesson 12 Teacher's Key: Hearing It Happen

Unity - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

Introducing a GameManager.instance singleton (mirroring Lesson 10's EnemySpawner.instance) specifically so static methods can reach an Inspector-assigned AudioSource, then using PlayOneShot() with three interchangeable clips for the round-ending sounds - genuinely new Unity technique, directly motivated by a real limitation (static methods can't see non-static Inspector fields without one).

Level 1 - Full Worked Solution

- GameManager.instance is new this week - if it didn't already exist for some other reason, this is the first time GameManager itself needs the singleton pattern, for exactly the reason EnemySpawner needed it in Lesson 10.
- sfx_shoot and sfx_hit stay exactly where they were (Player's and Enemy prefab's own AudioSource components, from Lessons 3-4) - only the round-ending sounds get the new shared roundAudioSrc treatment.
- PlayOneShot() (mentioned as a concept back in Lesson 2's key notes but not used until now) is the right call here specifically because it doesn't require checking .isPlaying first and layers naturally - a good payoff for a concept introduced early but not needed until this lesson.

Why This Structure

- PlayOneShot() versus .Play(): .Play() (used for shoot/hit) replaces whatever the AudioSource was already playing, which is fine for sounds triggered from their own dedicated source; PlayOneShot() layers a clip on top without disturbing anything already playing, which is the right choice for a shared source that might be asked to play three different things.
- Routing win/wave/lose through one shared instance rather than three separate GameObjects keeps the audio setup proportional to the game's actual complexity - three mutually-exclusive outcomes don't need three permanently-idle AudioSources.

Level 2 / Level 3 Talking Points

- Ask students to trace why instance.roundAudioSrc would throw a NullReferenceException if Awake() hadn't run yet - a good concrete illustration of initialization order mattering, reinforcing why EnemySpawner.instance needed the same care in Lesson 10.

- For students comparing engines: Godot's RoundSound.stream-swap and Unity's roundAudioSrc.PlayOneShot() solve the identical problem (one shared player, several possible outcomes) with genuinely different underlying tools - worth naming that cross-engine parallel explicitly.

Common Student Mistakes

- Missing or misplaced instance = this; in Awake(), causing a null reference the first time any RegisterKill()/RegisterEscape() call tries to reach roundAudioSrc.
- Placing the waveClip PlayOneShot() call outside the else branch, causing it to also fire on the true win.
- Forgetting to drag the AudioClip assets onto GameManager's Inspector fields after adding them in code - a common 'it compiles but plays nothing' mistake.